**Name – Winjeet Singh**

**Registration No – N/A**

**Module Name - Applied AI**

---

**Aim – 2. Car Price Prediction Dashboard**

**Problem:**

**Develop an interface to predict used car prices using features like year, fuel type, and mileage.**

**Task:**

**1. Load and clean car dataset.**

**2. Perform regression modeling.**

**3. Visualize feature importance.**

**4. Create user input interface.**

**5. Display predicted price with charts.**

**Requirements - Hardware Requirements**

- **Processor: Intel Core i3 or higher**
- **RAM: 4 GB or above**
- **Storage: 500 MB free space**
- **Operating System: Windows, Linux, or macOS**

**Software Requirements**

- **Python 3.x**
- **Jupyter Notebook / Google Colab / VS Code**
- **Pandas, NumPy, scikit-learn, matplotlib, seaborn, streamlit, joblib, Kaggle hub**

**Steps –**

**Step 1: Install Required Libraries**

- Install all required packages.
- pip install pandas numpy scikit-learn matplotlib seaborn streamlit joblib kagglehub

**Step 2: Download and Load the Dataset**

- Download the CarDekho dataset using KaggleHub.
- Load the CSV file into a Pandas DataFrame.
- Display the first few rows.
- Check dataset shape and columns.

**Step 3: Clean the Dataset**

- Check for missing values.
- Remove duplicate records.
- Keep only relevant columns.
- Verify data types.

**Step 4: Perform Regression Modeling**

- Separate features (X) and target (y).
- Convert categorical variables into numerical form.
- Split data into training and testing sets.
- Train a regression model.
- Evaluate model performance.

**Step 5: Visualize Feature Importance**

- Extract feature importance from the trained model.
- Create a bar chart showing the most influential features.

**Step 6: Create User Input Interface**

- Build a Streamlit dashboard.
- Add input fields:
    - Year
    - Present Price
    - Kilometers Driven
    - Fuel Type
    - Seller Type
    - Transmission
    - Owner

**Step 7: Display Predicted Price with Charts**

- Accept user input.
- Predict car price using the trained model.
- Display the predicted price.
- Show charts for:
    - Predicted price
    - Feature importance

**Code –**

**Cell 1: Import libraries**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import kagglehub
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error
```

**Cell 2: Download and load the dataset**

```
path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")
print("Dataset downloaded at:", path)
```

```
csv_files = list(Path(path).rglob("*.csv"))
print("CSV files found:", csv_files)

df = pd.read_csv(csv_files[0])
df.head()

Cell 3: Explore the dataset
print("Shape:", df.shape)

print("\nColumns:")
print(df.columns)
print("\nInfo:")
print(df.info())
print("\nMissing values:")
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())

Cell 4: Clean the dataset
df = df.drop_duplicates()

required_cols = [
  "Year",
  "Present_Price",
```

```
    "Kms_Driven",
    "Fuel_Type",
    "Seller_Type",
    "Transmission",
    "Owner",
    "Selling_Price"
]

df = df[required_cols]
df = df.dropna(subset=["Selling_Price"])

print("Shape after cleaning:", df.shape)
df.head()
Cell 5: Split features and target
X = df.drop("Selling_Price", axis=1)
y = df["Selling_Price"]

X.head(), y.head()

Cell 6: Prepare preprocessing
numeric_features = ["Year", "Present_Price", "Kms_Driven", "Owner"]
categorical_features = ["Fuel_Type", "Seller_Type", "Transmission"]
```

```
numeric_transformer = Pipeline(steps=[
   ("imputer", SimpleImputer(strategy="median"))
])

categorical_transformer = Pipeline(steps=[
   ("imputer", SimpleImputer(strategy="most_frequent")),
   ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
   transformers=[
      ("num", numeric_transformer, numeric_features),
      ("cat", categorical_transformer, categorical_features)
   ]
)

Cell 7: Train the regression model
model = RandomForestRegressor(n_estimators=200, random_state=42)

pipeline = Pipeline(steps=[
   ("preprocessor", preprocessor),
   ("model", model)
])
```

```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

pipeline.fit(X_train, y_train)

Cell 8: Evaluate the model
predictions = pipeline.predict(X_test)

r2 = r2_score(y_test, predictions)
mae = mean_absolute_error(y_test, predictions)

print(f"R2 Score: {r2:.4f}")
print(f"MAE: {mae:.4f}")

Cell 9: Save the trained model
joblib.dump(pipeline, "car_price_model.pkl")
print("Saved model to car_price_model.pkl")

Cell 10: Visualize feature importance
feature_names =
pipeline.named_steps["preprocessor"].get_feature_names_out()
```

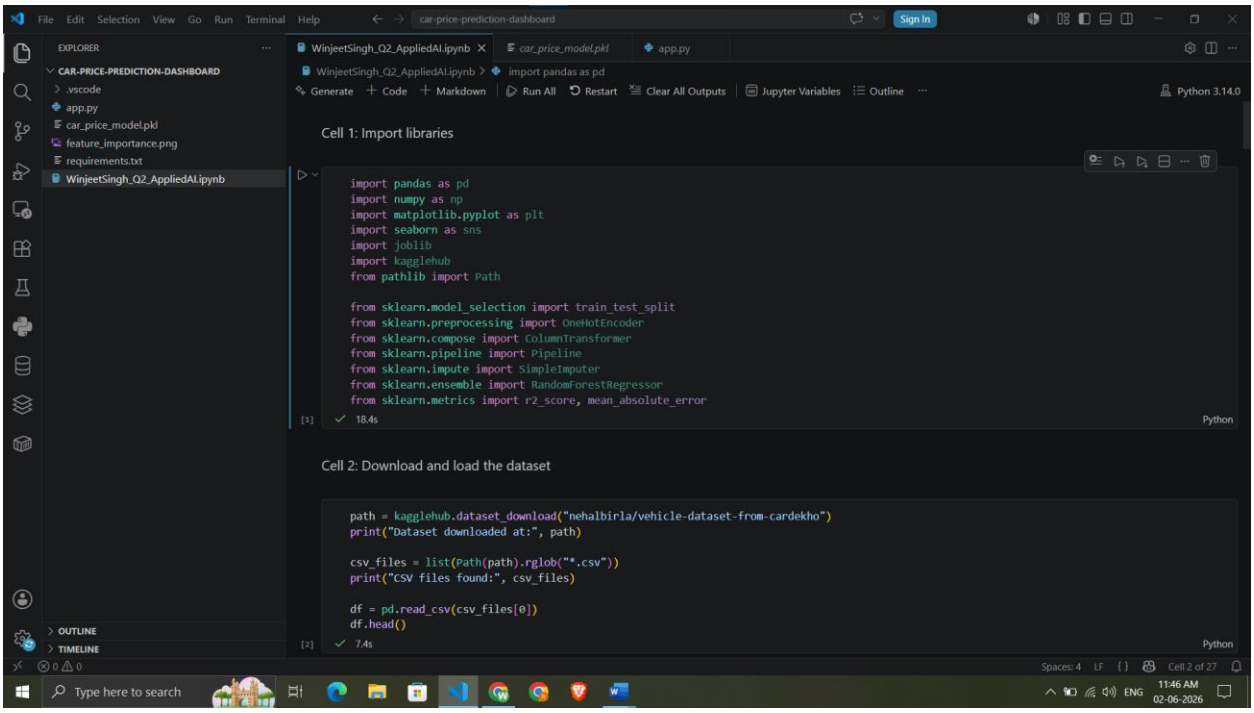
```
importances = pipeline.named_steps["model"].feature_importances_

importance_df = pd.DataFrame({
   "Feature": feature_names,
   "Importance": importances
}).sort_values("Importance", ascending=False)

importance_df.head(10)
plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df.head(10), x="Importance", y="Feature")
plt.title("Top Feature Importances")
plt.tight_layout()
plt.show()

Cell 11: Save feature importance chart
plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df.head(10), x="Importance", y="Feature")
plt.title("Top Feature Importances")
plt.tight_layout()
plt.savefig("feature_importance.png")
plt.show()
```

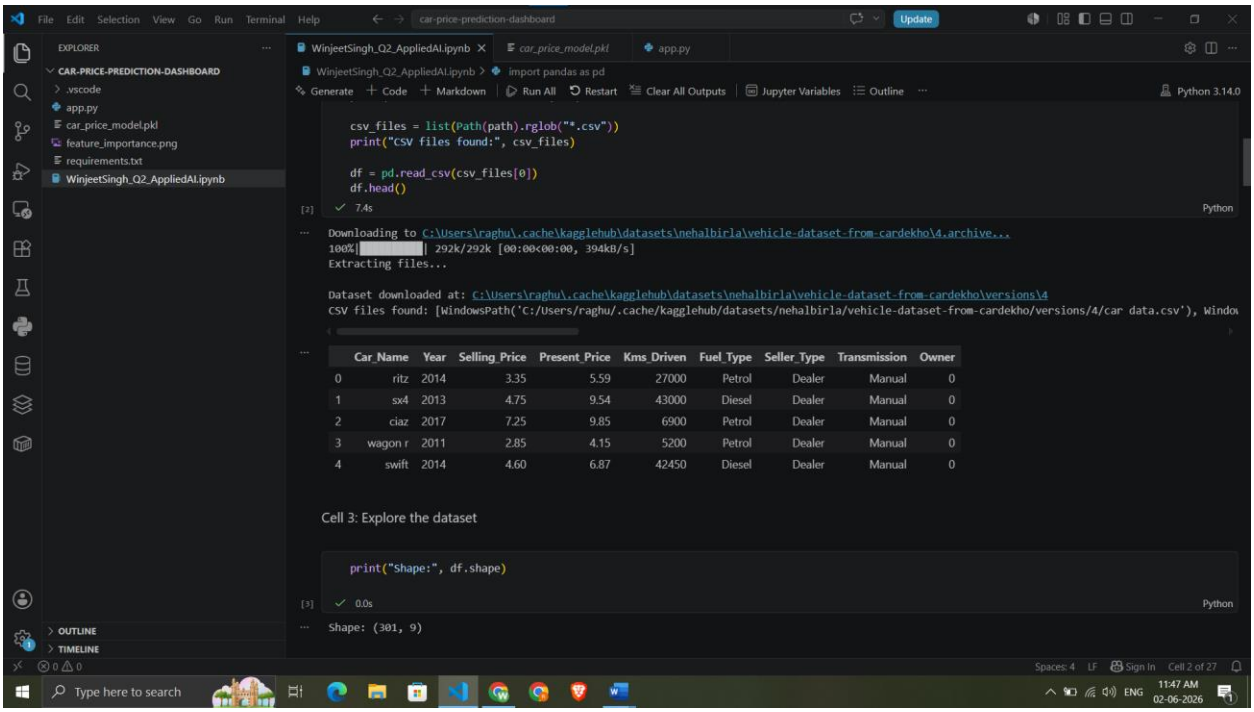
```
import streamlit as st
import pandas as pd
import joblib
import matplotlib.pyplot as plt

# Load model
model = joblib.load("car_price_model.pkl")

st.title("Car Price Prediction Dashboard")
st.write("Enter the car details below to predict the selling price.")

# User input interface
year = st.number_input("Year", min_value=1990, max_value=2026, value=2017)
present_price = st.number_input("Present Price", min_value=0.0, value=5.0, step=0.1)
kms_driven = st.number_input("Kms Driven", min_value=0, value=30000, step=500)
fuel_type = st.selectbox("Fuel Type", ["Petrol", "Diesel", "CNG"])
seller_type = st.selectbox("Seller Type", ["Dealer", "Individual"])
transmission = st.selectbox("Transmission", ["Manual", "Automatic"])
owner = st.number_input("Owner", min_value=0, max_value=10, value=0, step=1)
```

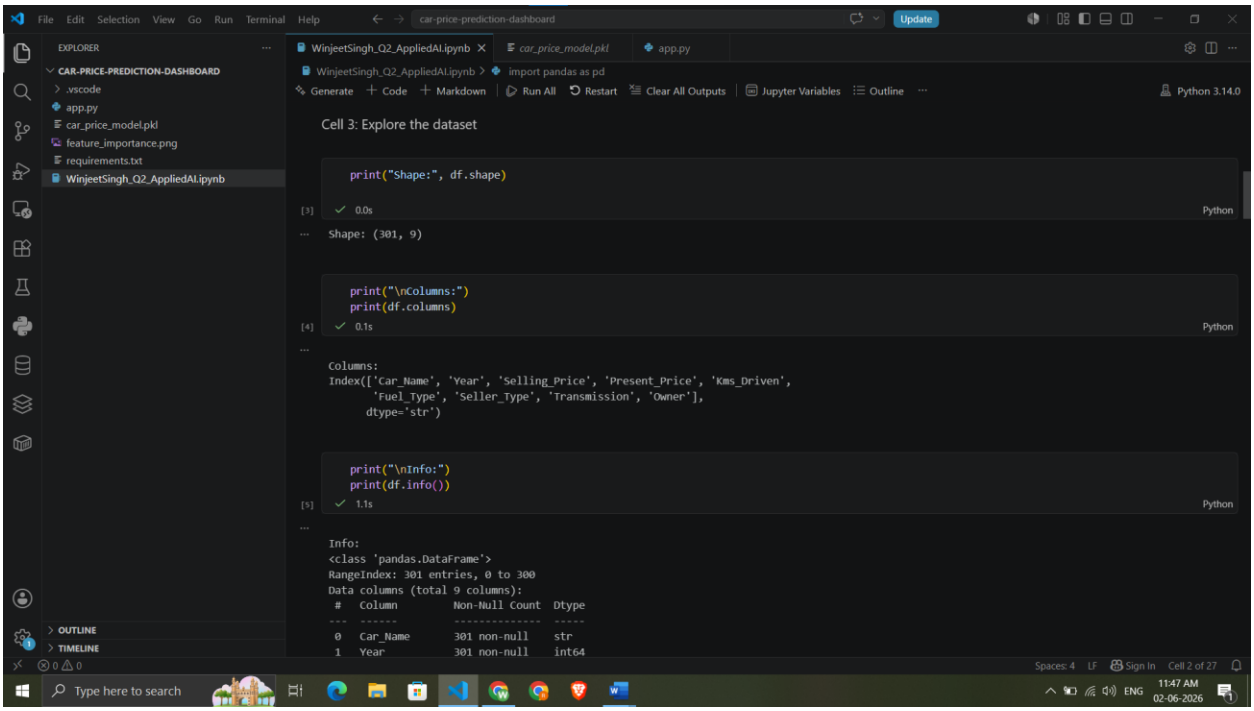
```
input_data = pd.DataFrame([{
    "Year": year,
    "Present_Price": present_price,
    "Kms_Driven": kms_driven,
    "Fuel_Type": fuel_type,
    "Seller_Type": seller_type,
    "Transmission": transmission,
    "Owner": owner
}])

if st.button("Predict Price"):
    prediction = model.predict(input_data)[0]

    st.success(f"Predicted Selling Price: ₹ {prediction:.2f} Lakhs")

    # Chart 1: predicted price
    st.subheader("Predicted Price Chart")
    price_df = pd.DataFrame({"Predicted Price": [prediction]})
    st.bar_chart(price_df)

    # Chart 2: feature importance
    st.subheader("Feature Importance")
```

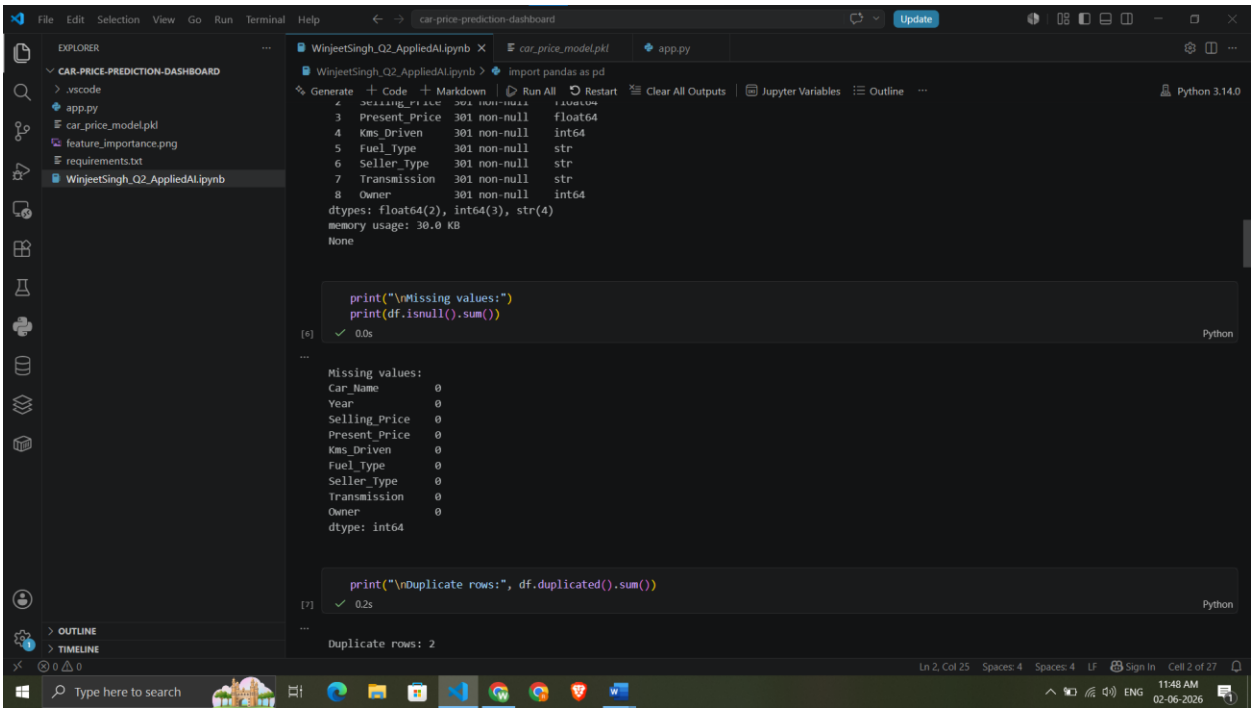
```
    feature_names =
model.named_steps["preprocessor"].get_feature_names_out()
    importances = model.named_steps["model"].feature_importances_

    importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Importance": importances
    }).sort_values("Importance", ascending=True).tail(10)

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.barh(importance_df["Feature"], importance_df["Importance"])
    ax.set_xlabel("Importance")
    ax.set_title("Top Feature Importances")
    st.pyplot(fig)
```

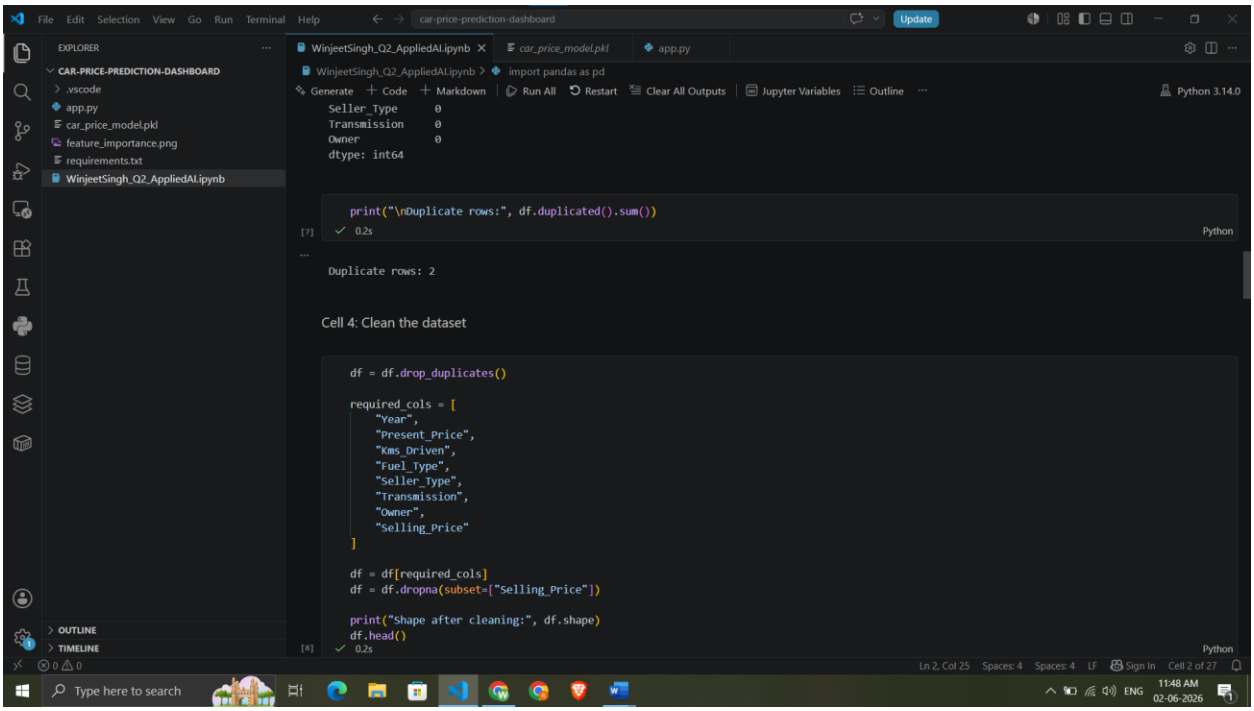
## Output Screenshot-

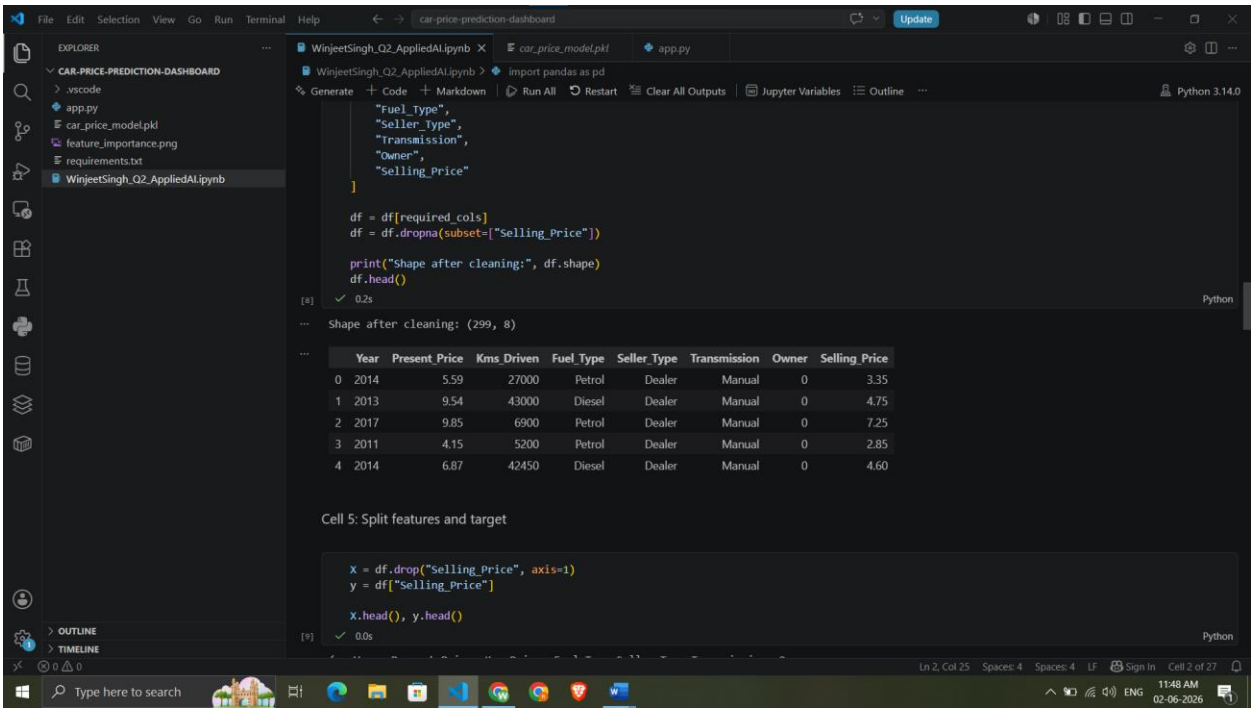


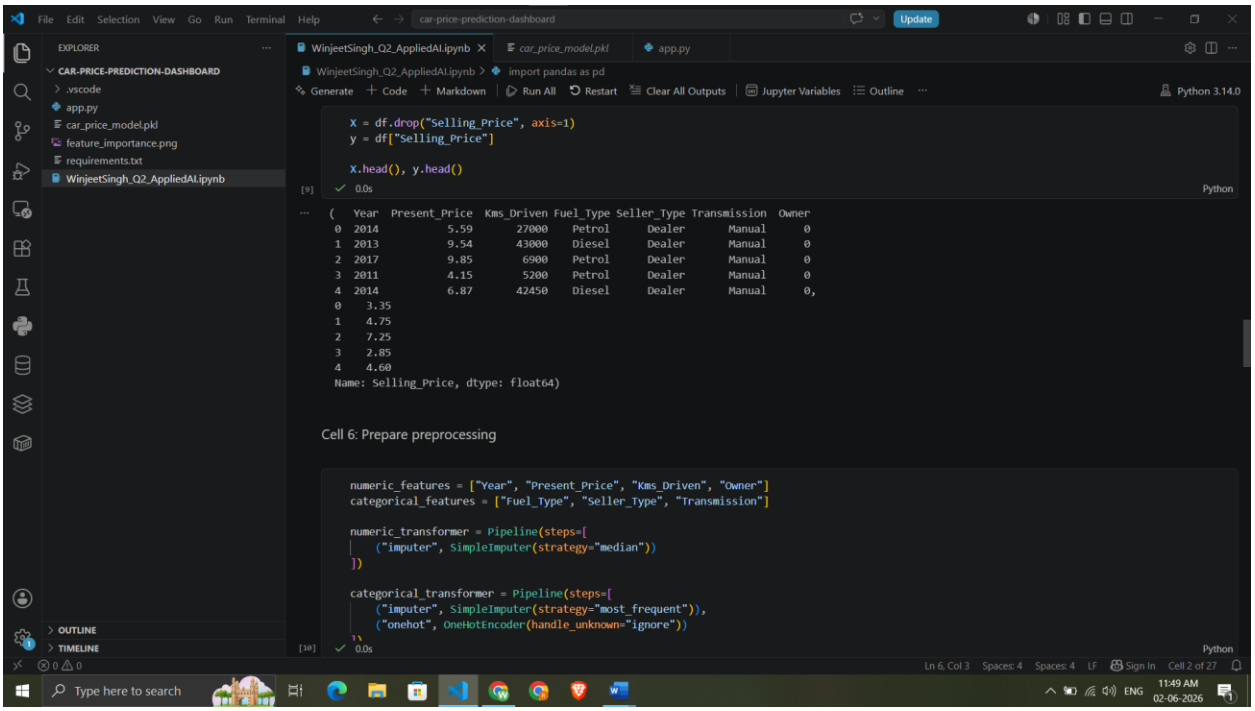



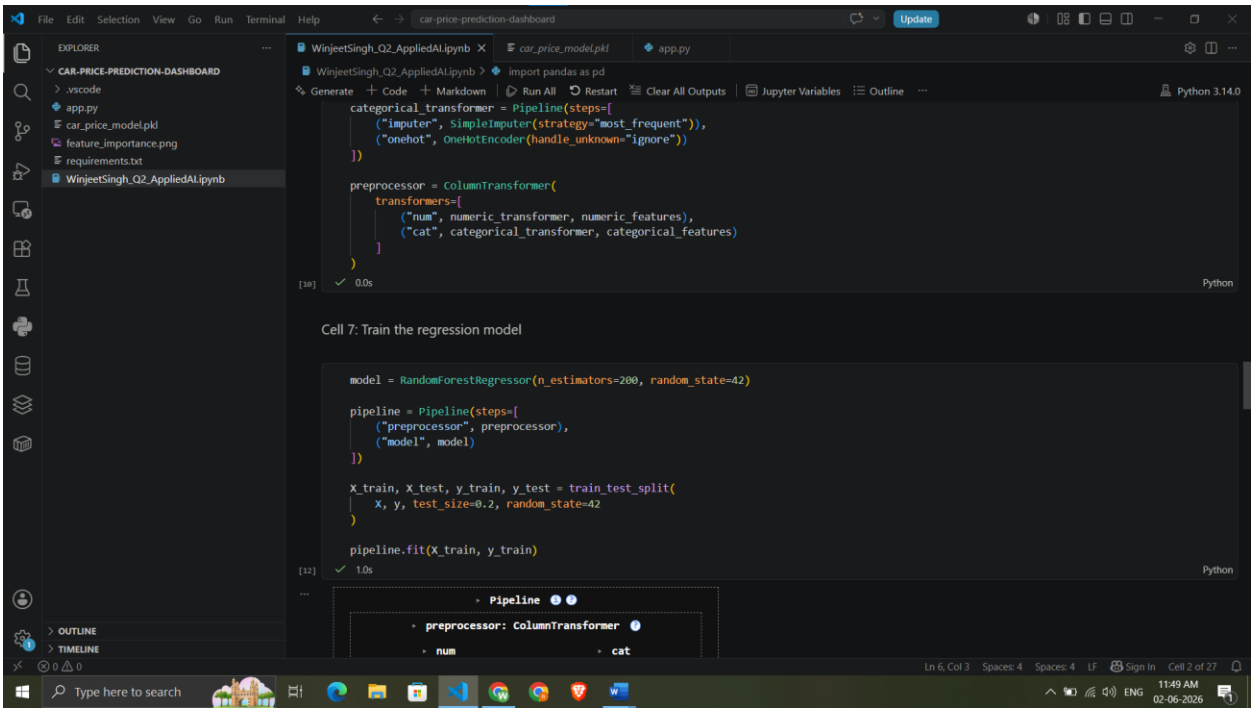

Cell 3: Explore the dataset
print("Shape:", df.shape)
Shape: (301, 9)
print("\nColumns:")
print(df.columns)
Columns:
Index(['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven',
'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner'],
dtype='str')
print("\nInfo:")
print(df.info())
Info:
<class 'pandas.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
# Column Non-Null Count Dtype
0 Car_Name 301 non-null str
1 Year 301 non-null int64

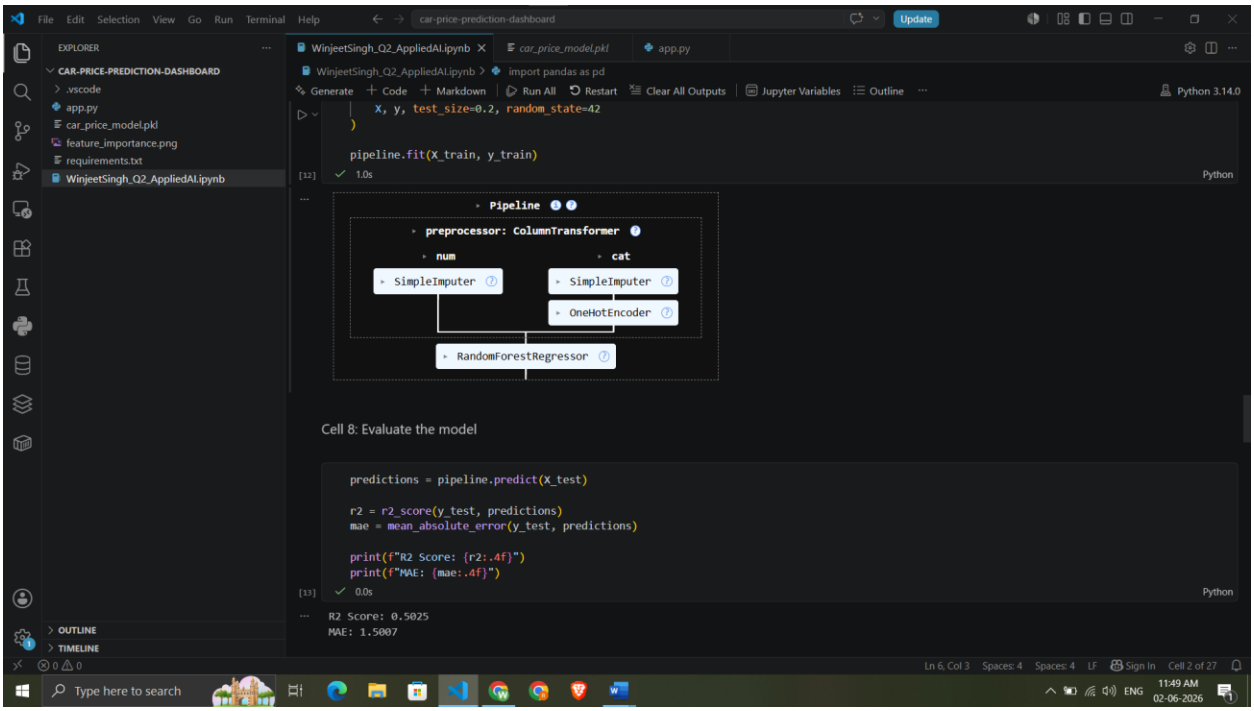

3 Present_Price 301 non-null float64
4 Kms_Driven 301 non-null int64
5 Fuel_Type 301 non-null str
6 Seller_Type 301 non-null str
7 Transmission 301 non-null str
8 Owner 301 non-null int64
dtypes: float64(2), int64(3), str(4)
memory usage: 30.0 KB
None
print("\nMissing values:")
print(df.isnull().sum())
Missing values:
Car_Name 0
Year 0
Selling_Price 0
Present_Price 0
Kms_Driven 0
Fuel_Type 0
Seller_Type 0
Transmission 0
Owner 0
dtype: int64
print("\nDuplicate rows:", df.duplicated().sum())
Duplicate rows: 2

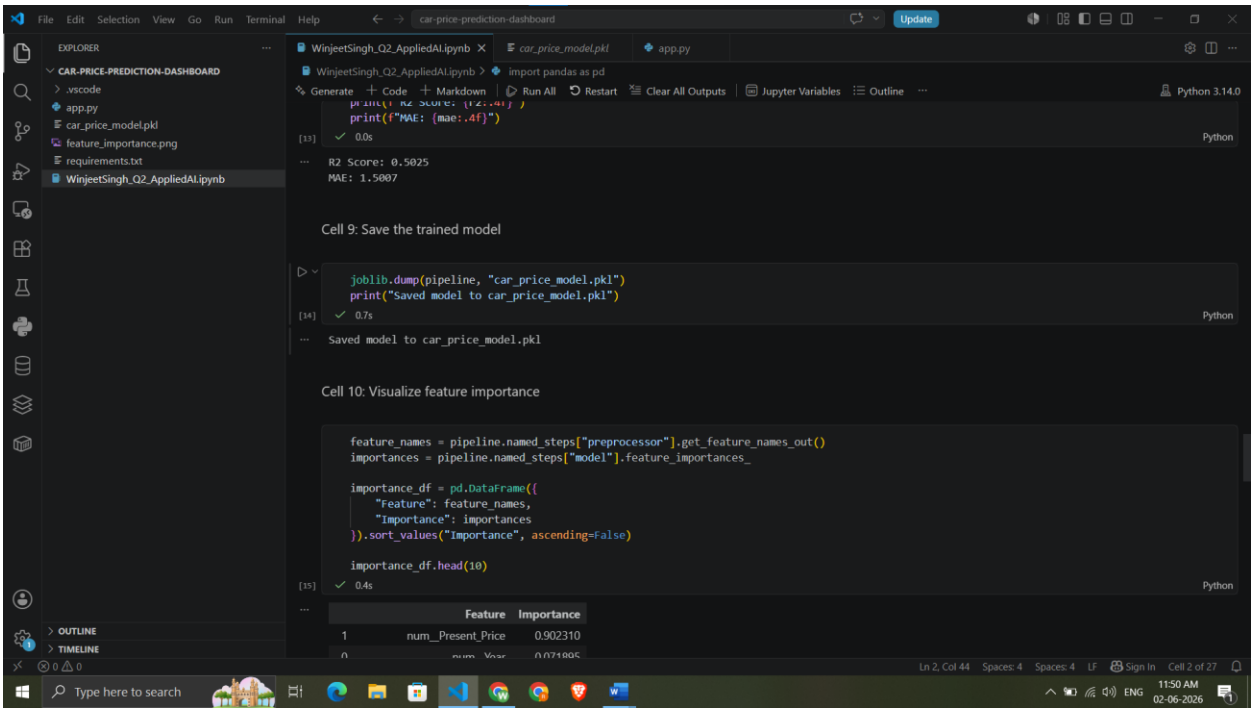

Seller_Type 0
Transmission 0
Owner 0
dtype: int64
print("\nDuplicate rows:", df.duplicated().sum())
Duplicate rows: 2
Cell 4: Clean the dataset
df = df.drop_duplicates()
required_cols = [
"Year",
"Present_Price",
"Kms_Driven",
"Fuel_Type",
"Seller_Type",
"Transmission",
"Owner",
"Selling_Price"
]
df = df[required_cols]
df = df.dropna(subset=["Selling_Price"])
print("Shape after cleaning:", df.shape)
df.head()

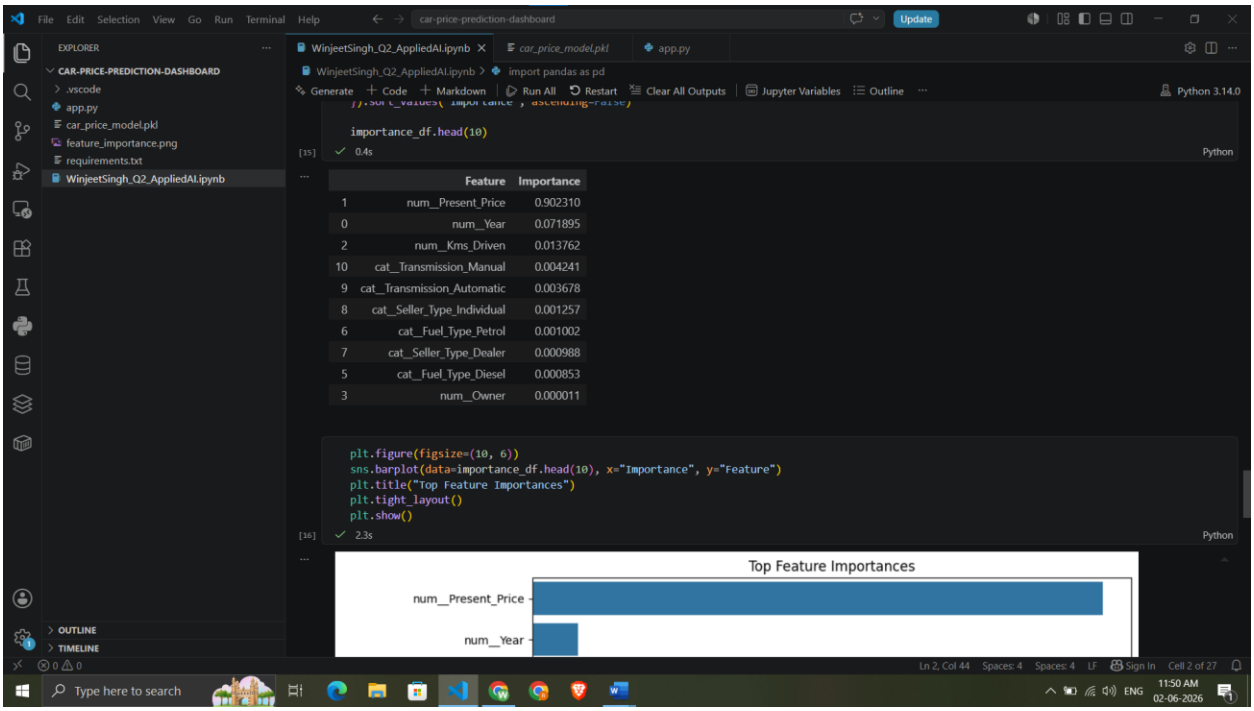

Shape after cleaning: (299, 8)
Year Present_Price Kms_Driven Fuel_Type Seller_Type Transmission Owner Selling_Price
0 2014 5.59 27000 Petrol Dealer Manual 0 3.35
1 2013 9.54 43000 Diesel Dealer Manual 0 4.75
2 2017 9.85 6900 Petrol Dealer Manual 0 7.25
3 2011 4.15 5200 Petrol Dealer Manual 0 2.85
4 2014 6.87 42450 Diesel Dealer Manual 0 4.60
Cell 5: Split features and target
X = df.drop("Selling_Price", axis=1)
y = df["Selling_Price"]
X.head(), y.head()

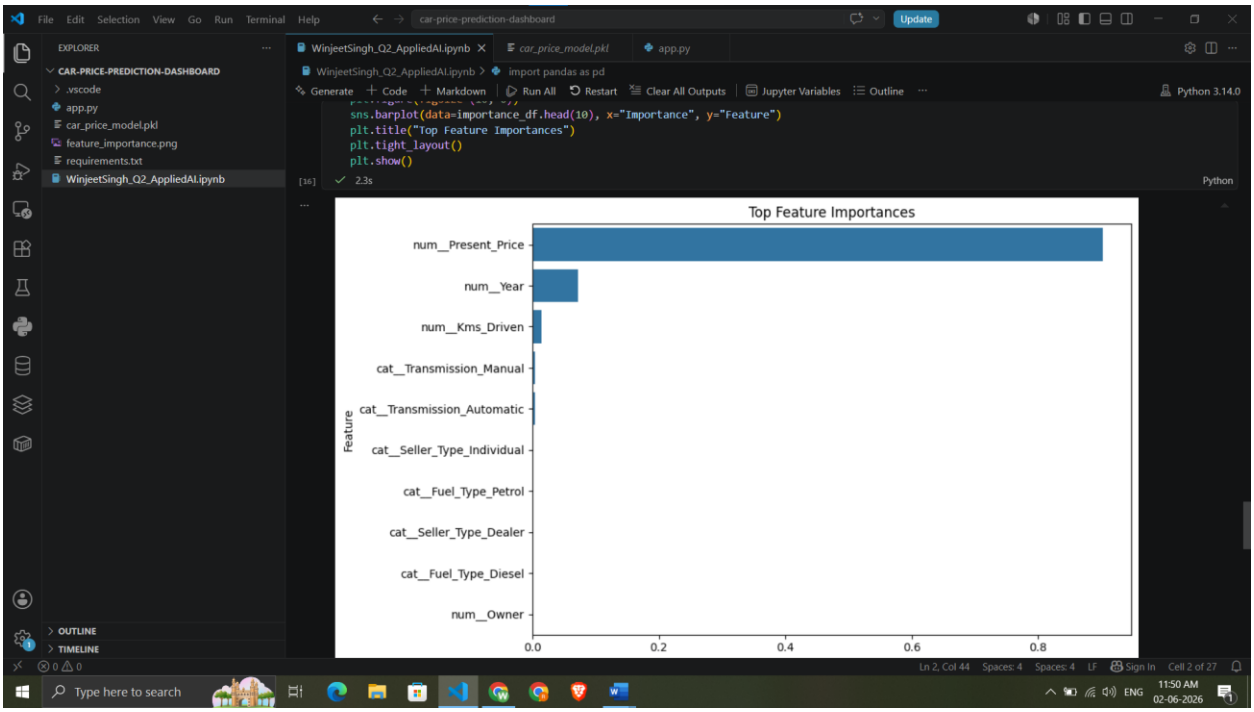

File Edit Selection View Go Run Terminal Help
car-price-prediction-dashboard
Update
EXPLORER
CAR-PRICE-PREDICTION-DASHBOARD
.vscode
app.py
car_price_model.pkl
feature_importance.png
requirements.txt
WinjeetSingh_Q2_AppliedAI.ipynb
WinjeetSingh_Q2_AppliedAI.ipynb
car_price_model.pkl
app.py
import pandas as pd
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.0
X = df.drop("Selling_Price", axis=1)
y = df["Selling_Price"]
X.head(), y.head()
[9]
0.0s
Python
( Year Present_Price Kms_Driven Fuel_Type Seller_Type Transmission Owner
0 2014 5.59 27000 Petrol Dealer Manual 0
1 2013 9.54 43000 Diesel Dealer Manual 0
2 2017 9.85 6900 Petrol Dealer Manual 0
3 2011 4.15 5200 Petrol Dealer Manual 0
4 2014 6.87 42450 Diesel Dealer Manual 0,
0 3.35
1 4.75
2 7.25
3 2.85
4 4.60
Name: Selling_Price, dtype: float64)
Cell 6: Prepare preprocessing
numeric_features = ["Year", "Present_Price", "Kms_Driven", "Owner"]
categorical_features = ["Fuel_Type", "Seller_Type", "Transmission"]
numeric_transformer = Pipeline(steps=[
("imputer", SimpleImputer(strategy="median"))
])
categorical_transformer = Pipeline(steps=[
("imputer", SimpleImputer(strategy="most_frequent")),
("onehot", OneHotEncoder(handle_unknown="ignore"))
])
[10]
0.0s
Python
OUTLINE
TIMELINE
Ln 6, Col 3 Spaces: 4 Spaces: 4 LF Sign In Cell 2 of 27
Type here to search
ENG
11:49 AM
02-06-2026

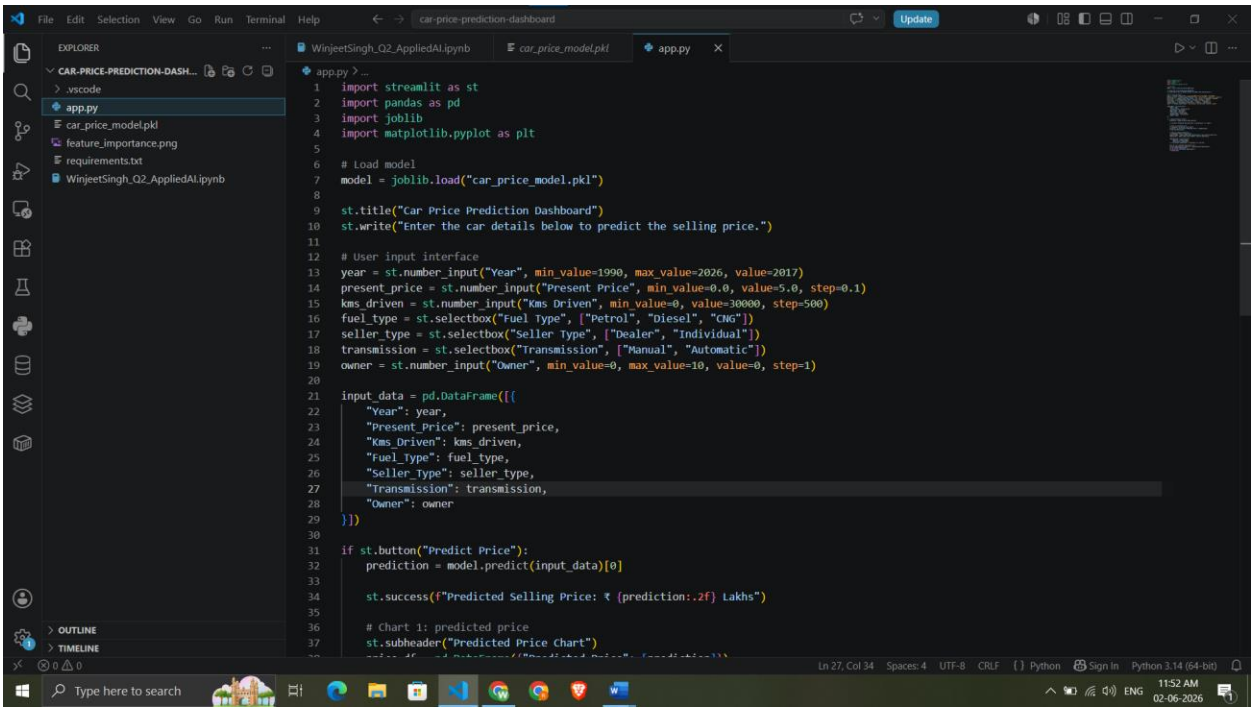

File Edit Selection View Go Run Terminal Help
car-price-prediction-dashboard
Update
EXPLORER
CAR-PRICE-PREDICTION-DASHBOARD
.vscode
app.py
car_price_model.pkl
feature_importance.png
requirements.txt
WinjeetSingh_Q2_AppliedAI.ipynb
WinjeetSingh_Q2_AppliedAI.ipynb
car_price_model.pkl
app.py
import pandas as pd
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.0
categorical_transformer = Pipeline(steps=[
("imputer", SimpleImputer(strategy="most_frequent")),
("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer(
transformers=[
("num", numeric_transformer, numeric_features),
("cat", categorical_transformer, categorical_features)
]
)
[10]
0.0s
Python
Cell 7: Train the regression model
model = RandomForestRegressor(n_estimators=200, random_state=42)
pipeline = Pipeline(steps=[
("preprocessor", preprocessor),
("model", model)
])
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
pipeline.fit(X_train, y_train)
[12]
1.0s
Python
Pipeline
preprocessor: ColumnTransformer
num
cat
OUTLINE
TIMELINE
Ln 6, Col 3 Spaces: 4 Spaces: 4 LF Sign In Cell 2 of 27
Type here to search
ENG
11:49 AM
02-06-2026


File Edit Selection View Go Run Terminal Help
car-price-prediction-dashboard
Update
EXPLORER
CAR-PRICE-PREDICTION-DASHBOARD
.vscode
app.py
car_price_model.pkl
feature_importance.png
requirements.txt
WinjeetSingh_Q2_AppliedAI.ipynb
WinjeetSingh_Q2_AppliedAI.ipynb
car_price_model.pkl
app.py
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.0
X, y, test_size=0.2, random_state=42
)
pipeline.fit(X_train, y_train)
[12]
1.0s
Python
Pipeline
preprocessor: ColumnTransformer
num
cat
SimpleImputer
SimpleImputer
OneHotEncoder
RandomForestRegressor
Cell 8: Evaluate the model
predictions = pipeline.predict(X_test)
r2 = r2_score(y_test, predictions)
mae = mean_absolute_error(y_test, predictions)
print(f"R2 Score: {r2:.4f}")
print(f"MAE: {mae:.4f}")
[13]
0.0s
Python
R2 Score: 0.5025
MAE: 1.5007
OUTLINE
TIMELINE
Ln 6, Col 3
Spaces: 4
Spaces: 4
LF
Sign In
Cell 2 of 27
Type here to search
ENG
11:49 AM
02-06-2026

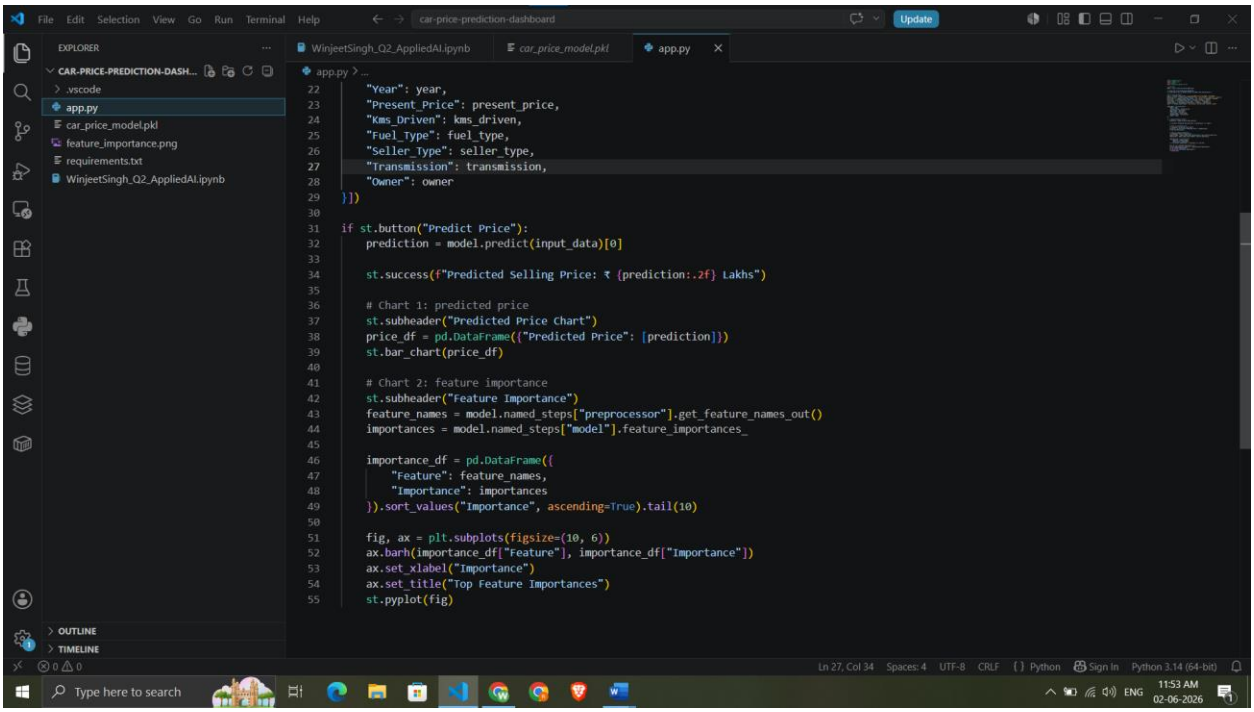

File Edit Selection View Go Run Terminal Help
car-price-prediction-dashboard
Update
EXPLORER
CAR-PRICE-PREDICTION-DASHBOARD
.vscode
app.py
car_price_model.pkl
feature_importance.png
requirements.txt
WinjeetSingh_Q2_AppliedAI.ipynb
WinjeetSingh_Q2_AppliedAI.ipynb
car_price_model.pkl
app.py
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.0
print(f"MAE: {mae:.4f}")
[13]
0.0s
Python
R2 Score: 0.5025
MAE: 1.5007
Cell 9: Save the trained model
joblib.dump(pipeline, "car_price_model.pkl")
print("Saved model to car_price_model.pkl")
[14]
0.7s
Python
Saved model to car_price_model.pkl
Cell 10: Visualize feature importance
feature_names = pipeline.named_steps["preprocessor"].get_feature_names_out()
importances = pipeline.named_steps["model"].feature_importances_
importance_df = pd.DataFrame({
"Feature": feature_names,
"Importance": importances
}).sort_values("Importance", ascending=False)
importance_df.head(10)
[15]
0.4s
Python
Feature
Importance
1
num__Present_Price
0.902310
OUTLINE
TIMELINE
Ln 2, Col 44
Spaces: 4
Spaces: 4
LF
Sign In
Cell 2 of 27
Type here to search
ENG
11:50 AM
02-06-2026


File Edit Selection View Go Run Terminal Help
car-price-prediction-dashboard
Update
EXPLORER
CAR-PRICE-PREDICTION-DASHBOARD
.vscode
app.py
car_price_model.pkl
feature_importance.png
requirements.txt
WinjeetSingh_Q2_AppliedAI.ipynb
WinjeetSingh_Q2_AppliedAI.ipynb
car_price_model.pkl
app.py
WinjeetSingh_Q2_AppliedAI.ipynb > import pandas as pd
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.14.0
importance_df.head(10)
[15] 0.4s
Python
Feature Importance
1 num__Present_Price 0.902310
0 num__Year 0.071895
2 num__Kms_Driven 0.013762
10 cat__Transmission_Manual 0.004241
9 cat__Transmission_Automatic 0.003678
8 cat__Seller_Type_Individual 0.001257
6 cat__Fuel_Type_Petrol 0.001002
7 cat__Seller_Type_Dealer 0.000988
5 cat__Fuel_Type_Diesel 0.000853
3 num__Owner 0.000011
plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df.head(10), x="Importance", y="Feature")
plt.title("Top Feature Importances")
plt.tight_layout()
plt.show()
[16] 2.3s
Python
Top Feature Importances
num__Present_Price
num__Year
OUTLINE
TIMELINE
Ln 2, Col 44 Spaces: 4 Spaces: 4 LF Sign In Cell 2 of 27
Type here to search
ENG
11:50 AM
02-06-2026

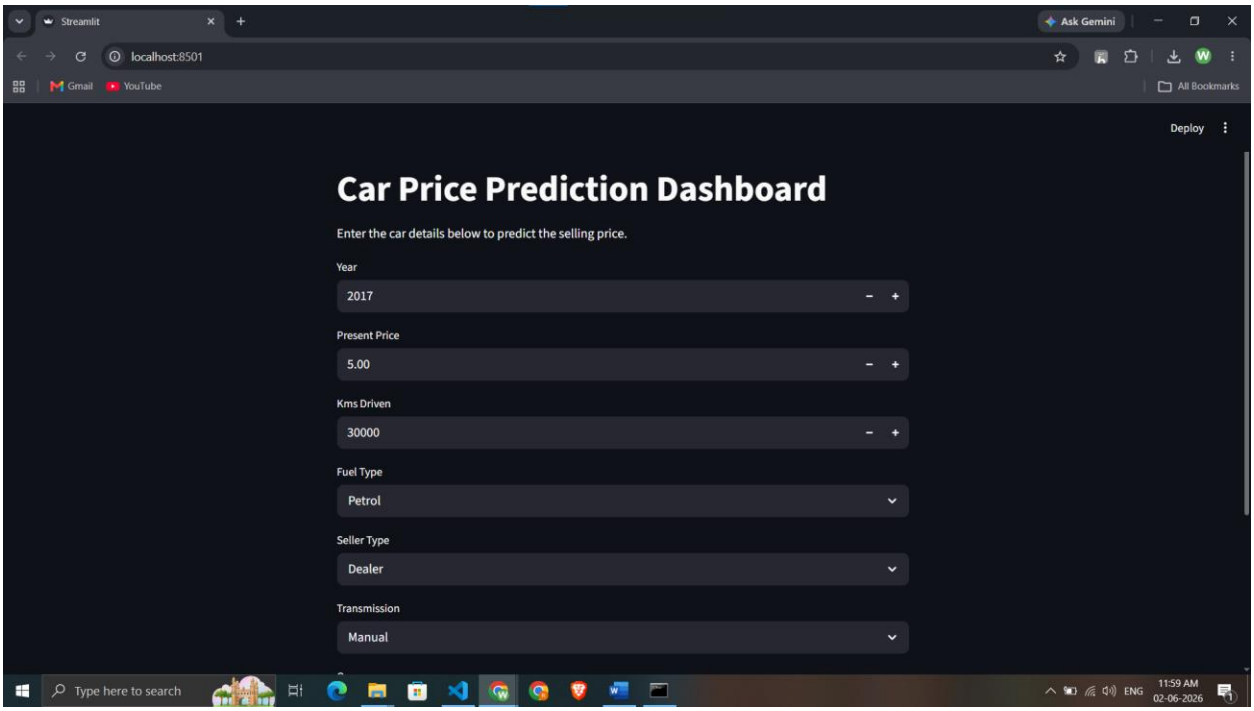

File Edit Selection View Go Run Terminal Help
car-price-prediction-dashboard
Update
EXPLORER
CAR-PRICE-PREDICTION-DASHBOARD
.vscode
app.py
car_price_model.pkl
feature_importance.png
requirements.txt
WinjeetSingh_Q2_AppliedAI.ipynb
WinjeetSingh_Q2_AppliedAI.ipynb
car_price_model.pkl
app.py
WinjeetSingh_Q2_AppliedAI.ipynb > import pandas as pd
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.14.0
sns.barplot(data=importance_df.head(10), x="Importance", y="Feature")
plt.title("Top Feature Importances")
plt.tight_layout()
plt.show()
[16] 2.3s
Python
Top Feature Importances
num__Present_Price
num__Year
num__Kms_Driven
cat__Transmission_Manual
cat__Transmission_Automatic
cat__Seller_Type_Individual
cat__Fuel_Type_Petrol
cat__Seller_Type_Dealer
cat__Fuel_Type_Diesel
num__Owner
Feature
0.0
0.2
0.4
0.6
0.8
OUTLINE
TIMELINE
Ln 2, Col 44 Spaces: 4 Spaces: 4 LF Sign In Cell 2 of 27
Type here to search
ENG
11:50 AM
02-06-2026

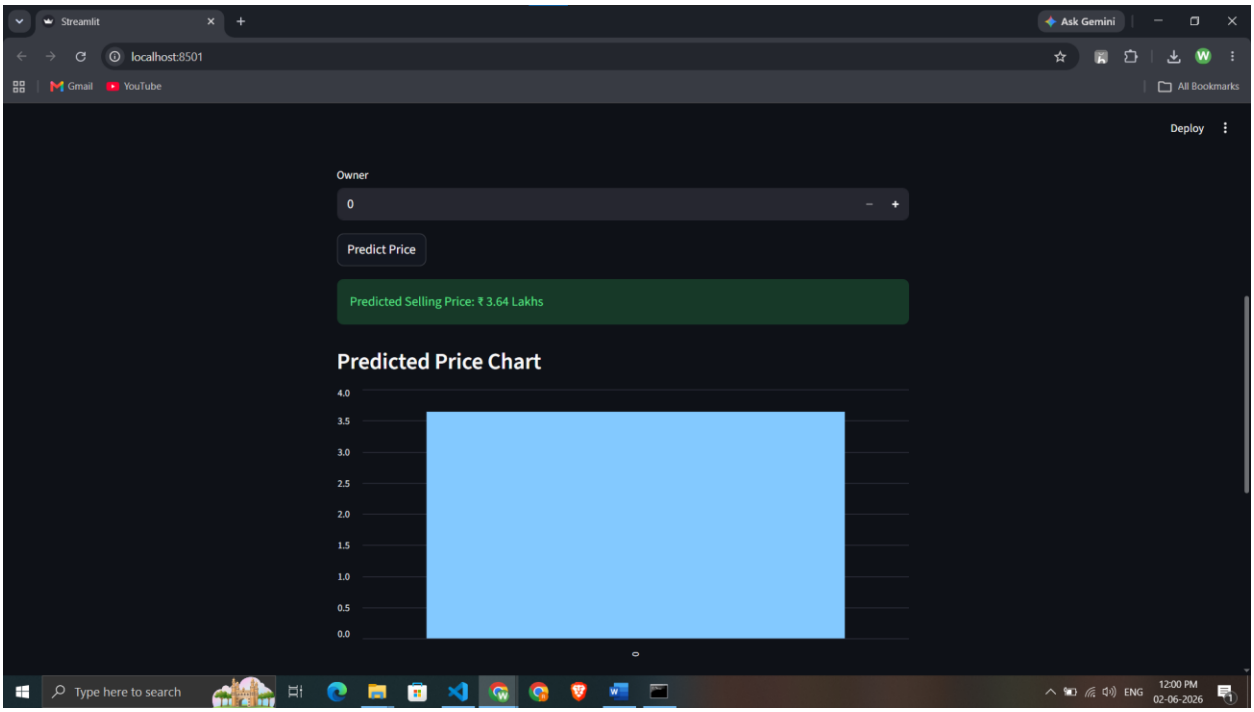


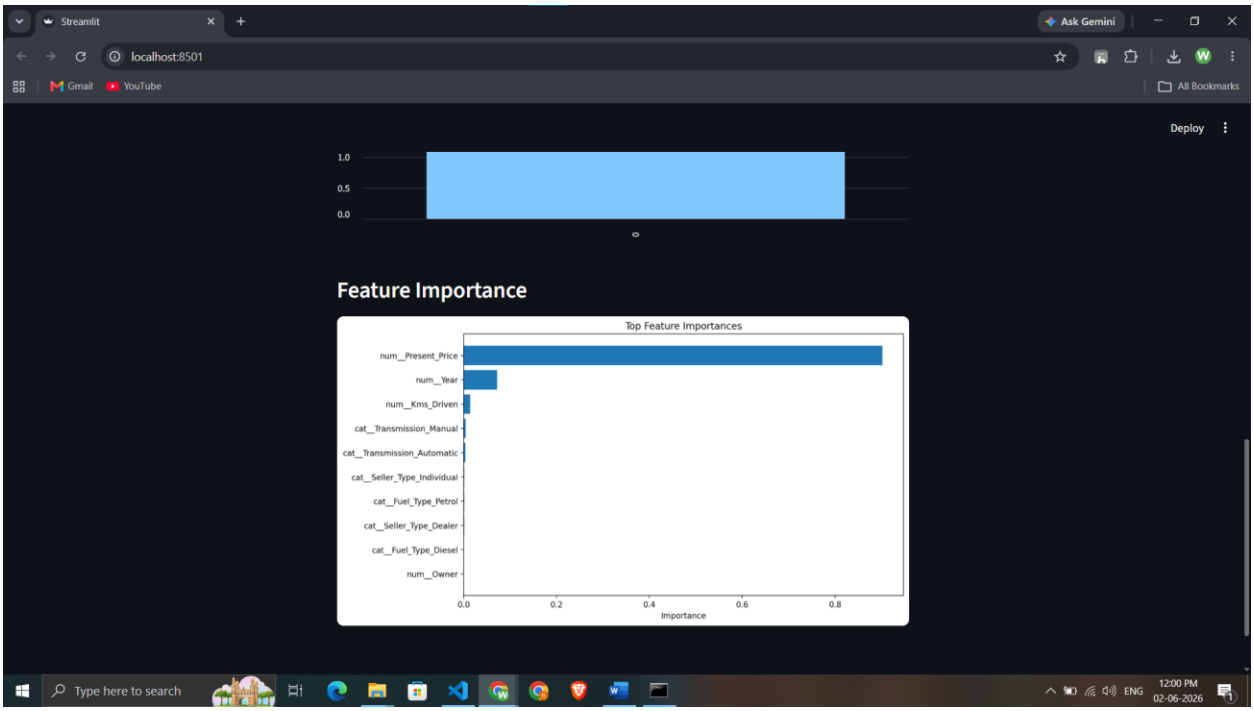
```python
import streamlit as st
import pandas as pd
import joblib
import matplotlib.pyplot as plt

# Load model
model = joblib.load("car_price_model.pkl")

st.title("Car Price Prediction Dashboard")
st.write("Enter the car details below to predict the selling price.")

# User input interface
year = st.number_input("Year", min_value=1990, max_value=2026, value=2017)
present_price = st.number_input("Present Price", min_value=0.0, value=5.0, step=0.1)
kms_driven = st.number_input("Kms Driven", min_value=0, value=30000, step=500)
fuel_type = st.selectbox("Fuel Type", ["Petrol", "Diesel", "CNG"])
seller_type = st.selectbox("Seller Type", ["Dealer", "Individual"])
transmission = st.selectbox("Transmission", ["Manual", "Automatic"])
owner = st.number_input("Owner", min_value=0, max_value=10, value=0, step=1)

input_data = pd.DataFrame([{
    "Year": year,
    "Present_Price": present_price,
    "Kms_Driven": kms_driven,
    "Fuel_Type": fuel_type,
    "Seller_Type": seller_type,
    "Transmission": transmission,
    "Owner": owner
}])

if st.button("Predict Price"):
    prediction = model.predict(input_data)[0]

    st.success(f"Predicted Selling Price: ₹ {prediction:.2f} Lakhs")

    # Chart 1: predicted price
    st.subheader("Predicted Price Chart")
```



```python
    "Year": year,
    "Present_Price": present_price,
    "Kms_Driven": kms_driven,
    "Fuel_Type": fuel_type,
    "Seller_Type": seller_type,
    "Transmission": transmission,
    "Owner": owner
}])

if st.button("Predict Price"):
    prediction = model.predict(input_data)[0]

    st.success(f"Predicted Selling Price: ₹ {prediction:.2f} Lakhs")

    # Chart 1: predicted price
    st.subheader("Predicted Price Chart")
    price_df = pd.DataFrame({"Predicted Price": [prediction]})
    st.bar_chart(price_df)

    # Chart 2: feature importance
    st.subheader("Feature Importance")
    feature_names = model.named_steps["preprocessor"].get_feature_names_out()
    importances = model.named_steps["model"].feature_importances_

    importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Importance": importances
    }).sort_values("Importance", ascending=True).tail(10)

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.barh(importance_df["Feature"], importance_df["Importance"])
    ax.set_xlabel("Importance")
    ax.set_title("Top Feature Importances")
    st.pyplot(fig)
```


Streamlit
Ask Gemini
localhost:8501
Gmail
YouTube
All Bookmarks
Deploy
Car Price Prediction Dashboard
Enter the car details below to predict the selling price.
Year
2017
Present Price
5.00
Kms Driven
30000
Fuel Type
Petrol
Seller Type
Dealer
Transmission
Manual
Type here to search
ENG
11:59 AM
02-06-2026


Streamlit
Ask Gemini
localhost:8501
Gmail
YouTube
All Bookmarks
Deploy
Owner
0
Predict Price
Predicted Selling Price: ₹ 3.64 Lakhs
Predicted Price Chart
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
Type here to search
ENG
12:00 PM
02-06-2026


Streamlit
Ask Gemini
localhost:8501
Gmail
YouTube
All Bookmarks
Deploy
1.0
0.5
0.0
0
Feature Importance
Top Feature Importances
num__Present_Price
num__Year
num__Kms_Driven
cat__Transmission_Manual
cat__Transmission_Automatic
cat__Seller_Type_Individual
cat__Fuel_Type_Petrol
cat__Seller_Type_Dealer
cat__Fuel_Type_Diesel
num__Owner
0.0
0.2
0.4
0.6
0.8
Importance
Type here to search
ENG
12:00 PM
02-06-2026